

A Deep Research Report on 3D Irregular Object Bin Packing for Python-Based Applications

Foundational Concepts and Computational Complexity of 3D Packing

The problem of packing three-dimensional objects into a confined space is a cornerstone of operations research, computer science, and industrial engineering. When dealing with irregular, undefined solid objects within a rectangular box, the challenge transcends traditional logistics and enters the realm of computational geometry and advanced algorithmic design. The fundamental objective, as specified by the user, is to maximize the number of such objects that can fit into a container of fixed size, while allowing for random orientation. This specific formulation is known as the Three-Dimensional Bin Packing Problem (3DBPP) with free rotation³. It is widely recognized in academic literature as being strongly NP-hard^{13 36}. This classification signifies that no known algorithm can solve every instance of the problem in polynomial time, and it is believed that such an algorithm does not exist. Consequently, the computational complexity of the problem grows exponentially with the number of items to be packed. For instance, a shipment containing just 50 items has more potential arrangement combinations than there are atoms in the observable universe, highlighting the immense difficulty of finding an optimal solution through brute-force enumeration³¹.

The NP-hardness of the problem dictates that practical solutions rely on heuristic and metaheuristic methods rather than exact algorithms capable of guaranteeing optimality for large-scale instances²². Heuristics are problem-specific rules of thumb designed to find a "good enough" solution quickly. Metaheuristics are higher-level strategies that guide a sub-problem-specific heuristic to efficiently explore the search space. These approaches form the backbone of modern bin packing software and research¹⁴. While these methods do not promise perfection, they have proven effective in real-world applications, enabling significant improvements in operational efficiency. Companies implementing such optimization systems often report substantial gains, including 12-18% reductions in shipping costs and 25-35% improvements in warehouse utilization within the first year of deployment³¹. Furthermore, achieving a packing density that reaches 85-90% of the theoretical lower bound is considered an excellent result in practice³¹. The core of the challenge lies in navigating a vast and complex search space defined by the position, orientation, and sequence of placement for each object.

This complexity is compounded by the nature of the objects themselves. The user's request for handling "irregular, undefined 3D objects" implies that the input will be in standard geometric file formats like STL or STEP^{1 9}. These files typically represent objects as a mesh of triangles, which can describe highly complex, non-convex, and hollow shapes. Traditional bin packing algorithms were developed for simpler, rectangular items, where orthogonally aligning them with the container axes is sufficient¹³. However, for irregular shapes, this restriction is too limiting. Allowing for

continuous rotation provides a much larger solution space, which theoretically leads to higher packing densities but also makes the problem computationally far more demanding ³. This is because the search must now consider not only the x, y, z coordinates for placement but also two rotational degrees of freedom (e.g., yaw and pitch), significantly increasing the dimensionality of the search. The problem becomes even more intricate when considering practical constraints beyond pure volume maximization, such as load stability, weight distribution, fragility, and loading sequence ^{22 36}. For example, ensuring that each placed object has adequate support from below and is not top-heavy is critical for preventing damage during transport. Some advanced algorithms address this by incorporating physics-based heuristics, such as verifying that the center of mass of an object lies within the convex hull of its contact points with other objects or the base ²⁷. Thus, the task at hand is not merely a mathematical exercise in volume filling but a multi-faceted optimization problem that requires balancing competing objectives within a constrained computational budget.

Geometric Representation and Mesh Processing in Python

A successful implementation of a 3D irregular object packing solver hinges on the ability to accurately and efficiently process the geometric representations of the objects. Given the user's requirement for Python-based tools and support for common file formats like STL and STEP, the choice of libraries is critical. Several powerful open-source Python libraries are available for this purpose, each with distinct strengths. The most prominent library for handling triangular meshes is Trimesh. Trimesh is a pure Python library that excels at loading, analyzing, and visualizing meshes from various formats, including .obj, STL, and PLY ²⁶. Its key advantages include robust support for essential geometric analysis tasks. For instance, it can compute the volume (`mesh.volume`), check if a mesh is watertight (`mesh.is_watertight`), calculate the center of mass, and generate the convex hull of a mesh (`mesh.convex_hull`) ²⁶. These functionalities are indispensable for any packing algorithm, as they provide the fundamental geometric properties needed for collision detection and placement evaluation. Another C++-based library with high-performance Python bindings is Open3D, which offers similar capabilities for mesh and point cloud processing, including loading meshes (`o3d.io.read_triangle_mesh`), computing normals, and applying transformations ²⁶. For more general-purpose CAD scripting and manipulation, especially with STEP files, CadQuery is an excellent choice. Built on top of the Open CASCADE Technology (OCCT) kernel, CadQuery is a parametric CAD framework that supports both importing and exporting STEP and STL files, making it suitable for pre-processing CAD models before packing ⁹.

However, simply loading a mesh is not always sufficient. Many sophisticated packing algorithms operate on simplified geometric representations to improve performance. One common technique is convex decomposition, which breaks down a complex, non-convex object into a set of convex components ^{3 35}. Convex shapes have well-defined geometric properties that make collision detection and other calculations much faster and more stable. For example, one study explicitly discretized non-convex components into convex sub-components to facilitate their dynamic simulation approach ³⁵. Another technique is voxelization, which involves converting the continuous surface of a mesh into a discrete 3D grid of cubic cells, or voxels ^{5 17}. Voxelization simplifies the geometry and enables the use of highly efficient algorithms, such as those based on Fast Fourier Transform (FFT), for tasks like collision detection ^{3 17}. Tools like **binvox** can be used to convert STL files into a voxel representation ⁵. Libraries like PySLM utilize Trimesh for mesh handling and support voxelization as part of its workflow ¹⁶. While these approximations are necessary for

scalability, they introduce a trade-off between computational speed and accuracy. The chosen level of approximation must be carefully balanced against the desired precision of the final packing arrangement.

Beyond basic loading and transformation, Python libraries offer advanced features crucial for the packing process. For instance, Trimesh can split a mesh into its constituent connected components (`mesh.split()`), which is useful if a single file contains multiple separate objects ²⁶. The `numpy-stl` library, another key tool, focuses specifically on STL files and provides functions for reading, writing, and modifying them ¹. It leverages numpy for performance, making it one of the fastest STL editing libraries in Python ¹. It can also compute important physical properties like volume, center of gravity, and inertia tensor directly from the mesh data ¹. Other libraries cater to different needs; PyMesh supports advanced polygon clipping via the Clipper2 library and includes modules for constructive solid geometry (CSG) operations like union and difference ³⁰. SolidPython allows users to generate OpenSCAD code programmatically, which is useful for creating precise CAD models from parametric descriptions ³⁰. For GPU-accelerated processing, which can be beneficial for very large or complex scenes, PyTorch3D is a library that leverages GPU power for mesh operations, although it may require careful memory management ³⁰. The selection of these tools provides a comprehensive toolkit for the entire preprocessing pipeline, from ingesting raw CAD data to generating the simplified geometric representations required by the core packing algorithm.

Library	Primary Focus	Key Strengths	Supported Formats	Relevant Functionality
Trimesh	Pure Python mesh analysis and visualization	Watertightness checking, convex hull computation, volume calculation, vertex/face manipulation ²⁶	STL, OBJ, PLY, 3MF (read-only) ²⁶	<code>mesh.volume</code> , <code>mesh.convex_hull</code> , <code>mesh.split()</code> ²⁶
Open3D	High-performance 3D data processing (C++ backend)	Fast mesh loading, normal computation, integration with ML frameworks ²⁶	STL, OBJ, PLY, XYZ, PNG ²⁶	<code>o3d.io.read_triangle_mesh</code> , visualization with animation ²⁶
CadQuery	Parametric CAD scripting	Strong STEP/STL import/export, OCCT-based	STEP, STP, STL, BREP, IGES ⁹	Importing CAD files, geometric modification ⁹

Library	Primary Focus	Key Strengths	Supported Formats	Relevant Functionality
	(Python-native)	modeling kernel ⁹		
numpy-stl	Efficient STL file handling using NumPy	Fast read/write, mass property calculation (volume, COM) ¹	Binary and ASCII STL ¹	<code>mesh.area</code> , <code>mesh.center_of_mass</code> , <code>mesh.rotate()</code> ¹
PySLM	Additive manufacturing (SLM/DMLS) prep	Slicing, hatching, support generation, overhang analysis ¹⁶	STL ¹⁶	Slicing triangular meshes, build-time estimation ¹⁶

Core Algorithms for Maximizing Packing Density

Achieving the goal of maximizing the number of irregular objects in a box requires moving beyond simple greedy placement heuristics. The most effective approaches for this NP-hard problem fall into several categories: physics-based simulations, metaheuristics, and hybrid deep reinforcement learning (DRL) methods. Each category offers a unique philosophy for navigating the vast solution space. Physics-based simulations model the packing process as a physical system. One such method is the dynamic acceleration methodology, which simulates the motion of objects under the influence of forces designed to achieve a dense packing state ³⁵. In this approach, objects are subjected to forces that resolve overlaps, attract them towards a central region, align their rotational axes, and reduce their overall energy. The simulation runs until the system settles into a low-energy configuration, which ideally corresponds to a tightly packed arrangement. This method has demonstrated impressive results, achieving average packing densities of 64% with spheres, 95% with cubes, and over 95% with tetracube structures, showcasing its effectiveness for polyomino-like shapes ³⁵. An alternative physics-inspired approach uses rigid body dynamics simulation, where objects are dropped into a container and subjected to periodic horizontal shaking to enable a self-assembly-like rearrangement that improves density ³. A key advantage of these methods is their conceptual simplicity and their ability to handle complex, free-form geometries without needing to pre-discretize orientations.

Metaheuristic algorithms provide a more structured, yet still exploratory, approach. These methods combine a global search strategy with local improvement procedures. A notable example is the hybrid chaos firefly algorithm (HFA), which was applied to the three-dimensional irregular packing problem ³². This algorithm embeds a chaos search mechanism into the standard firefly algorithm to enhance population diversity and avoid premature convergence to suboptimal solutions. The HFA optimizes both the sequence in which objects are placed and their orientations. By allowing for

continuous rotation, this approach achieved packing height reductions of 1.82% to 4.80% compared to non-optimized placements, with further improvements of 1.35% to 2.32% when rotation was enabled ³². Another powerful metaheuristic is the hybrid metaheuristic LNSGA (Local Neighborhood Search Genetic Algorithm), which combines a genetic algorithm with the Gurobi MIP solver ^{6 18}. This hybrid approach leverages the global exploration strength of the genetic algorithm to find promising regions of the search space and then uses the precise, local optimization capabilities of the Gurobi solver to refine the solutions found. LNSGA has been shown to outperform both Gurobi and a standard genetic algorithm, achieving an average optimization improvement of 50.16% over Gurobi on large-scale instances while reducing runtime by over 50% compared to the GA alone ⁶. Such hybrid methods represent the cutting edge of metaheuristic research, effectively combining the best of evolutionary and mathematical programming techniques.

Deep Reinforcement Learning (DRL) represents a paradigm shift, treating the packing problem as a sequential decision-making process. DRL agents learn an optimal policy for placing objects by interacting with a simulated environment and receiving feedback in the form of rewards. A prominent example is the GOPT (General Online Packing Transformer) model, which is a Transformer-based DRL approach designed for online 3D bin packing ²³. GOPT uses a Placement Generator module to identify viable placement candidates (free sub-spaces) and a Packing Transformer to fuse item and bin features to predict the optimal placement. While currently limited to cuboid items with restricted orientations, GOPT achieves a space utilization of 76.1%, outperforming previous state-of-the-art baselines ²³. Similarly, the HHPPO (Hybrid Heuristic and Deep Proximal Policy Optimization) method integrates a deterministic heuristic with a DRL agent trained via PPO ¹¹. This hybrid approach achieved 92% maximum space utilization in one test scenario, a 7% increase over a baseline, and successfully deployed on a real robot manipulator ¹¹. A key feature of some DRL models is the incorporation of physics heuristics to ensure stability; for example, one model introduced a "convexHull-k" heuristic that verifies an object's stability by checking if its center of mass is supported, drastically reducing object fall rates from 5.1% to just 0.03% during placement ²⁷. These DRL approaches are particularly promising because they can generalize across different bin sizes and unseen item types, offering a degree of flexibility that traditional algorithms lack ²³. The choice of algorithm depends on the specific requirements regarding solution quality, computational time, and the need for generalization.

Collision Detection and Constraint Management

For any packing algorithm involving free rotation and arbitrary placement, the most computationally intensive and critical operation is collision detection. The algorithm must constantly and accurately determine whether a proposed placement of an object would cause it to overlap with the container walls, the floor, or other already-placed objects. Without an efficient method for this check, the entire packing process becomes intractable. The primary challenge stems from the fact that the objects are represented by complex triangle meshes, and a naive pairwise comparison of every triangle in the new object against every triangle in all existing objects would have a complexity of $O(N^2)$, which is prohibitively slow for any meaningful number of objects ². To overcome this, researchers employ a variety of acceleration structures and algorithms.

A foundational concept is the use of Bounding Volumes. Instead of testing the complex mesh itself, the algorithm first tests a much simpler shape that completely encloses the mesh, such as an Axis-

Aligned Bounding Box (AABB) or a tighter-fitting Oriented Bounding Box (OBB)^{7 8}. Since testing AABBs is trivial, this provides a fast way to eliminate pairs of objects that are clearly too far apart to be colliding. This initial step is often referred to as the "broad phase" of collision detection⁸. The next step, the "narrow phase," involves performing a more precise, and more expensive, check only on the pairs of objects that survived the broad-phase test. For triangle meshes, this involves primitive-primitive tests like segment vs. triangle or triangle vs. triangle intersection¹⁰. For convex shapes, a highly efficient algorithm is the Gilbert – Johnson – Keerthi (GJK) distance algorithm^{2 3}⁷. GJK can determine the minimum distance between two convex shapes and whether they are intersecting, making it ideal for collision detection. It is often combined with the Expanding Polytope Algorithm (EPA) to compute the penetration depth and vector when a collision occurs³⁵. The use of BVHs (Bounding Volume Hierarchies), such as OBB trees, organizes these bounding volumes in a tree structure, allowing for efficient culling of large groups of objects that cannot possibly be colliding^{7 8}.

These techniques are not merely theoretical; they are implemented in numerous software libraries and are central to the functioning of major physics engines like Bullet and Havok². The performance gains from using these optimized methods are dramatic. One study showed that using quadrees for spatial partitioning reduced processing time by 97% compared to a brute-force pair-by-pair check². In the context of the user's project, the choice of collision detection strategy is paramount. If the goal is to reduce packaging time, then investing in a robust, optimized collision detection module is essential. The dynamic acceleration method, for example, relies heavily on GJK/EPA to resolve geometric overlaps in real-time during its simulation³⁵. Similarly, the voxel-based "nofit voxel" (NFV) approach forgoes direct mesh-mesh collision for a more abstract but computationally cheaper representation of forbidden placements⁵.

Beyond simple non-overlap, the user's application may need to manage other constraints. For instance, if the objects are fragile, the algorithm might need to ensure a minimum separation distance between them. If stability is a concern, it could enforce that each object has support from below, perhaps by requiring that its centroid lies above the convex hull of its contact points with other objects²⁷. Some problems incorporate "defective regions" or prohibited areas on the container floor, which can be handled by modifying the placement heuristics to avoid those zones¹². The integration of these constraints adds another layer of complexity to the optimization process. For example, the HHPPO method was able to integrate partial support constraints into its reward function, guiding the DRL agent to prioritize stable placements¹¹. Ultimately, the effectiveness of any packing algorithm is contingent on the efficiency and accuracy of its underlying collision detection and constraint management system.

Performance Metrics and Benchmark Analysis

Evaluating the success of a 3D irregular object packing algorithm requires a clear understanding of the relevant performance metrics and the benchmarks against which new methods are measured. The primary metric for the user's objective—maximizing the number of objects—is packing density, which is defined as the total volume of all packed objects divided by the volume of the container³. Higher packing density directly translates to a greater number of items fitting into the box. However, depending on the specific business case, other metrics may also be important. For instance, if items need to be accessed sequentially from the top, minimizing the height of the resulting stack might be a

secondary objective. In such cases, the packing height (the vertical extent of the packed items) becomes a key metric, as seen in the "irregular 3D open dimension problem" (I3DODP) ⁵. The choice of metric guides the design of the algorithm's objective function; for the user's problem, the objective is simply to maximize the number of items, which is equivalent to maximizing the cumulative volume of the packed subset of items.

Performance is typically benchmarked against established datasets and compared to the results of previously published algorithms. Academic research often uses standardized benchmark instances to ensure reproducibility and comparability. For example, one study on a voxel-based ILP and metaheuristic approach tested its methods on randomly generated 'blobs' instances and real-world 3D printing models, such as an 'Engine' model with 97 items and a 'Chess' model with 32 items ⁵. Another paper evaluating the HHPPO DRL method used a Model 1 with container dimensions of 400 cm × 300 cm × 200 cm and differently sized objects ¹¹. The LNSGA metaheuristic was tested on a dataset of 1,000 unique item sizes generated from SKU dimensions with lengths from 21-30, widths from 11-20, and heights from 1-10 ⁶. These benchmarks allow researchers to quantitatively assess how their new methods perform relative to the state of the art.

The table below summarizes the performance of several representative algorithms discussed in the provided sources, illustrating the range of results achievable.

Algorithm / Method	Problem Type	Key Features	Reported Performance Metric	Source
Dynamic Acceleration Method	Irregular Shapes, Free Rotation	Physics simulation, multiple forces (GOR, CA, etc.)	64% avg. density (12 spheres), 95% avg. density (8 cubes)	³⁵
Hybrid Chaos Firefly Algorithm (HFA)	Irregular Shapes, Variable Rotation	Optimizes packing sequence and orientations	Packing height reduction of 1.82-4.80% (w/o rot.), 3.17-7.12% (w/ rot.)	³²
GOPT (Transformer-based DRL)	Cuboid Items, Orthogonal Placements	Generalizes across bin sizes, uses physics heuristic	76.1% space utilization	²³
HHPPO (Heuristic + DRL)	Online 3D Bin Packing	Integrates extreme point priority sorting	92% max space utilization (7% increase over baseline)	¹¹
Variable Neighborhood Search (VNS)	Irregular 3D Open Dimension	Voxel-based ILP, metaheuristic	Up to 8% better average packing height on 'blobs' instances	⁵
LNSGA (Genetic Alg. + MIP)	Multiorder Rectangular Packing	Hybrid metaheuristic,	50.16% avg. improvement over	⁶

Algorithm / Method	Problem Type	Key Features	Reported Performance Metric	Source
		optimizes sequence & positions	Gurobi on large instances	
Skyline-based Algorithms	Irregular Shapes (approximated)	Bottom-left placement, mixed sizes	92-96% space utilization	22

As the table shows, performance varies significantly based on the problem specifics and the sophistication of the algorithm. Simple, approximation-based skyline algorithms can achieve very high space utilization (92-96%) but are less suited for truly irregular, free-rotating objects [22](#). More advanced methods tailored for irregular shapes yield lower absolute densities but are the only viable option for the user's stated problem. The hybrid chaos firefly algorithm achieves moderate but significant improvements over non-optimized placements [32](#), while the metaheuristic LNSGA demonstrates massive gains over commercial solvers on large, complex instances [6](#). The DRL-based GOPT and HHPPO methods show strong performance and the added benefit of generalization, though they currently focus on simpler-shaped items [11](#) [23](#). For the user, this analysis suggests that aiming for a packing density comparable to what is reported for irregular shape solvers (e.g., around 60-75%) is a realistic target, with the potential to exceed this with a sufficiently advanced metaheuristic or simulation-based approach.

Implementation Strategy and Toolchain Integration

To construct a robust Python-based solution for 3D irregular object bin packing, a coherent implementation strategy and a well-integrated toolchain are essential. The process can be broken down into a series of logical steps that transform raw geometric data into an optimized packing arrangement. The following outlines a recommended workflow based on the capabilities of the available libraries and the principles of effective packing algorithms.

Step 1: Data Ingestion and Preprocessing The initial stage involves loading the 3D object definitions from STL or STEP files. The recommended starting point is to use Trimesh or CadQuery due to their comprehensive support for these formats and their powerful mesh analysis capabilities [9](#) [26](#). Once loaded, the mesh should be analyzed for basic properties required for packing. This includes calculating the volume, checking if the mesh is watertight (a prerequisite for many algorithms), and potentially computing the convex hull to serve as a conservative, but faster-to-process, proxy for the original shape [26](#). If the objects are known to be non-convex and the chosen algorithm requires it, this is the stage to perform convex decomposition.

Step 2: Geometric Simplification and Representation Given the computational expense of working with detailed triangle meshes, especially for collision detection, a critical step is to simplify the geometry. The appropriate simplification depends entirely on the chosen algorithm. * For Physics-Based Simulation: The mesh can be used directly if the simulation engine (e.g., a custom-built one or a physics library like Bullet) can handle complex collision detection. * For Voxelization-Based Methods: The mesh should be converted into a voxel representation using a tool like **binvox** [5](#).

The resolution of this voxelization is a key parameter; a finer resolution provides more accuracy but increases memory usage and computation time. * For Metaheuristic Approaches: A simplified representation, such as a convex hull or a set of convex components from decomposition, is often used for the main optimization loop. The actual mesh can be retained for final visualization and precise placement verification. * For Deep Learning Models: The object representation would likely be a point cloud or voxel grid fed into the neural network.

Step 3: Algorithm Selection and Core Logic Implementation This is the heart of the project. Based on the user's goals, the most promising approaches are either a physics-based simulation or a metaheuristic optimizer. * **Physics Simulation Approach:** This involves creating a virtual environment where objects are subject to forces. A core loop would repeatedly: 1. Select an unplaced object. 2. Propose a random position and orientation within the container. 3. Apply forces to move and orient the object, resolving collisions with the container and other objects using an algorithm like GJK/EPA³⁵. 4. If the object comes to rest without violating constraints, place it permanently. 5. Repeat until no more objects can be placed stably. * **Metaheuristic Approach (e.g., LNSGA):** This requires defining a chromosome (a candidate solution). A typical chromosome could be a permutation of object indices representing the placement order, plus a list of orientations for each object. The fitness function would involve: 1. Initializing an empty container. 2. Iterating through the object order specified by the chromosome. 3. Attempting to place each object in its specified orientation using a placement heuristic (like Deepest-Bottom-Left-Fill, DBLF³) and checking for collisions. 4. The fitness value is the total number of successfully placed objects.

Step 4: Collision Detection Module Regardless of the main algorithm, a fast and reliable collision detection module is non-negotiable. This module should be built on optimized data structures like BVHs or use specialized algorithms like GJK^{2 7}. Python libraries like Trimesh have built-in collision detection routines, while PyBullet provides a full-featured physics engine that can be used for this purpose¹⁶. This module should be modular so it can be swapped out or enhanced independently.

Step 5: Integration and Execution The various components—the data loader, the geometric simplifier, the algorithm logic, and the collision detector—must be integrated into a cohesive program. For performance-critical tasks, leveraging libraries with C++ backends like Open3D or Trimesh is advisable²⁶. If the algorithm is expected to run for a long time, progress should be logged periodically. Finally, the output should be a list of successful placements (object index, position, orientation) that can be used for visualization or feeding into downstream processes like robotic picking.

In summary, the recommended toolchain consists of Trimesh or CadQuery for I/O and preprocessing, a voxelization tool like **binvox** if needed, and a physics engine like PyBullet or a custom implementation using GJK for the core simulation. For a metaheuristic approach, this would be complemented by a mathematical programming solver like Gurobi (if accessible) and a genetic algorithm library. This integrated strategy provides a powerful and flexible foundation for tackling the challenging problem of packing irregular 3D objects.

Reference

1. numpy-stl - PyPI <https://pypi.org/project/numpy-stl/>
2. How does 3D collision / object detection work? - Stack Overflow <https://stackoverflow.com/questions/1960560/how-does-3d-collision-object-detection-work>
3. Dynamics simulation-based packing of irregular 3D objects <https://www.sciencedirect.com/science/article/abs/pii/S0097849324001316>
4. The 3D bin packing problem for multiple boxes and irregular items ... https://www.researchgate.net/publication/372247223_The_3D_bin_packing_problem_for_multiple_boxes_and_irregular_items_based_on_deep_Q-network
5. Voxel-Based Solution Approaches to the Three-Dimensional ... <https://pubsonline.informs.org/doi/10.1287/opre.2022.2260>
6. A Genetic Algorithm with Lower Neighborhood Search for the Three ... <https://onlinelibrary.wiley.com/doi/10.1155/2024/4456261>
7. What are the commonly used collision detection techniques in 3D ... <https://www.tencentcloud.com/techpedia/100407>
8. 3D Collision Detection Library for Python and C++ - MeshLib <https://meshlib.io/feature/collision-detection/>
9. CadQuery/cadquery: A python parametric CAD scripting ... - GitHub <https://github.com/CadQuery/cadquery>
10. [PDF] Collision Detection https://cseweb.ucsd.edu/classes/wi17/cse169-a/slides/CSE169_12.pdf
11. Integrating Heuristic Methods with Deep Reinforcement Learning for ... <https://www.mdpi.com/1424-8220/24/16/5370>
12. Heuristic algorithms for the special knapsack packing problem with ... <https://www.sciencedirect.com/science/article/abs/pii/S0957417422024101>
13. The Three-Dimensional Bin Packing Problem | Operations Research <https://pubsonline.informs.org/doi/10.1287/opre.48.2.256.12386>
14. Optimizing 3D Irregular Object Packing from 3D Scans Using ... <https://www.sciencedirect.com/science/article/abs/pii/S1474034620302032>
15. Nesting or Packing in Grasshopper in 3D space with mesh objects <https://discourse.mcneel.com/t/nesting-or-packing-in-grasshopper-in-3d-space-with-mesh-objects/166297>
16. PySLM: A Python Library for 3D Printing and Additive Manufacturing <https://github.com/drlukeparry/pyslm>

17. The chore of packing just got faster and easier | MIT News <https://news.mit.edu/2023/chore-packing-just-got-faster-and-easier-0706>
18. A Genetic Algorithm with Lower Neighborhood Search for the Three ... <https://dl.acm.org/doi/10.1155/2024/4456261>
19. A heuristic for solving the irregular strip packing problem with ... - arXiv <https://arxiv.org/html/2402.17542v1>
20. A New Heuristic Algorithm for the 3D Bin Packing Problem https://www.researchgate.net/publication/226249396_A_New_Heuristic_Algorithm_for_the_3D_Bin_Packing_Problem
21. Automating the Packing Heuristic Design Process with Genetic ... <https://direct.mit.edu/evco/article/20/1/63/916/Automating-the-Packing-Heuristic-Design-Process>
22. How Box Packing Algorithms Save Costs | 3DBinPacking Blog <https://www.3dbinpacking.com/en/blog/box-packing-algorithms-space-optimization/>
23. GOPT: Generalizable Online 3D Bin Packing via Transformer-based ... <https://arxiv.org/html/2409.05344v2>
24. Three-stage heuristic algorithm for three-dimensional irregular ... <https://www.sciencedirect.com/science/article/pii/S0307904X16304887>
25. Python STL — Python STL dev documentation <https://python-stl.readthedocs.io/>
26. Python Libraries for Mesh, Point Cloud, and Data Visualization (Part 1) <https://towardsdatascience.com/python-libraries-for-mesh-and-point-cloud-visualization-part-1-daa2af36de30/>
27. An Efficient Deep Reinforcement Learning Model for Online 3D Bin ... <https://arxiv.org/html/2408.09694v1>
28. Is there a Python library to generate STL files for 3D printing? [closed] <https://stackoverflow.com/questions/23123384/is-there-a-python-library-to-generate-stl-files-for-3d-printing>
29. A global search framework for practical three-dimensional packing ... <https://www.sciencedirect.com/science/article/abs/pii/S0305054811003625>
30. 3D modeling with Python - Medium <https://medium.com/@alexeyyurasov/3d-modeling-with-python-c21296756db2>
31. Bin Packing Optimization That Works | 3DBinPacking Blog <https://www.3dbinpacking.com/en/blog/bin-packing-optimization-strategies/>
32. A hybrid chaos firefly algorithm for three-dimensional irregular ... <http://www.aims sciences.org/article/doi/10.3934/jimo.2018160>
33. Optimal three-dimensional particle shapes for maximally dense ... <https://pubs.aip.org/aip/jcp/article/161/1/014505/3300370/Optimal-three-dimensional-particle-shapes-for>
34. Machine learning approaches for the optimization of packing ... <https://pubs.rsc.org/en/content/articlehtml/2023/sm/d2sm01430k>

35. Packing optimization of practical systems using a dynamic ... <https://jeas.springeropen.com/articles/10.1186/s44147-024-00426-6>
36. Constrained-optimization in a 3D bin packing realistic problem https://www.researchgate.net/publication/355842855_Constrained-optimization_in_a_3D_bin_packing_realistic_problem